

LABORATORIO 10

Dos escritores al mismo tiempo

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

Introducción

1 · El tema

En este laboratorio vas a poner dos escritores sobre la misma tabla, al mismo tiempo y a propósito.

No es un caso raro. Es lo que pasa todas las noches en una plataforma de datos, cuando dos procesos de carga parten a la misma hora y tocan la misma tabla. La pregunta del laboratorio es qué pasa en ese momento, y la respuesta tiene dos partes que conviene no confundir.

2 · El problema, en el almacén

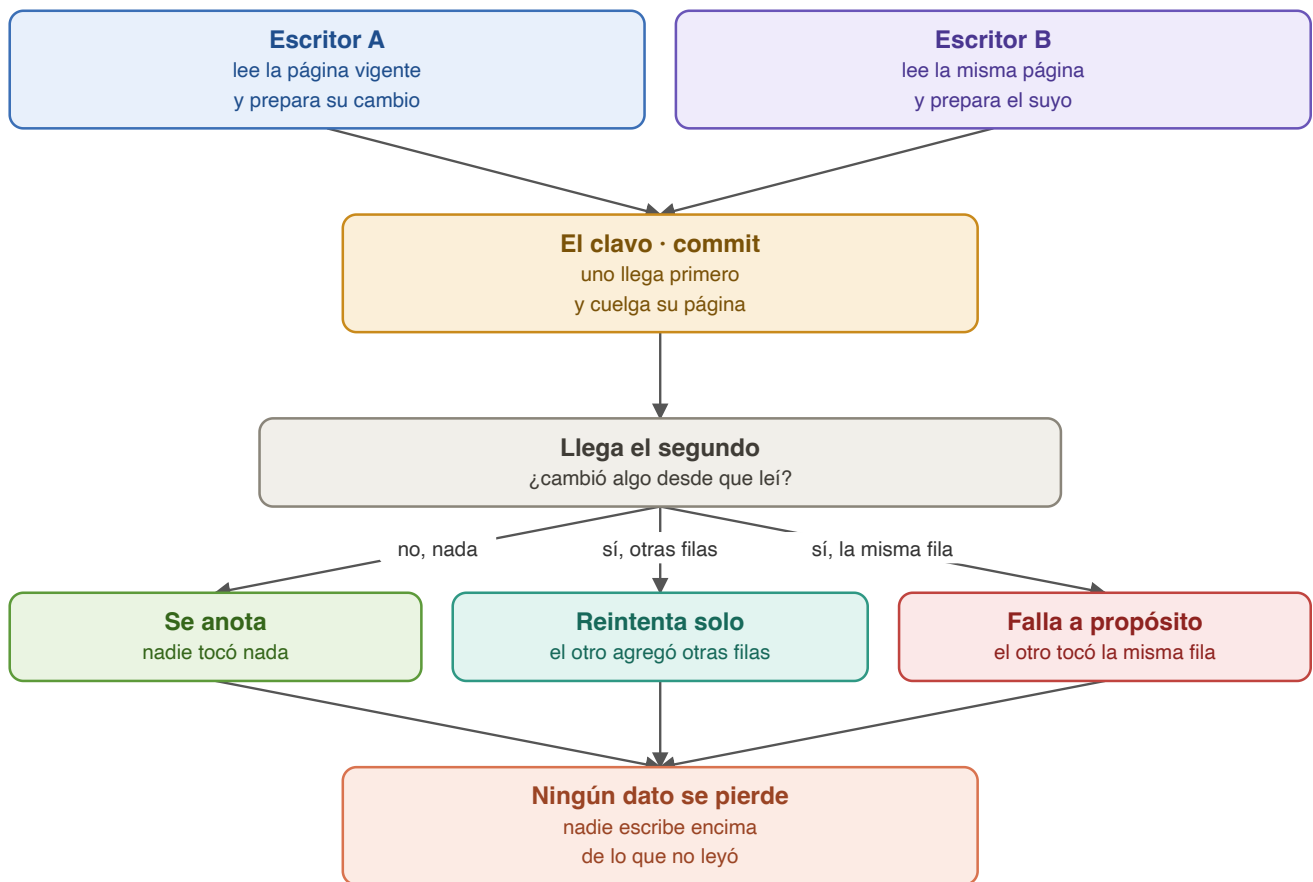
Dos personas quieren anotar en la misma libreta al mismo tiempo. Si cada una anota algo nuevo al final, no hay problema, porque caben las dos anotaciones. Pero si las dos quieren corregir el mismo renglón, sí hay problema. Una va a escribir encima de la otra, y la primera corrección se pierde sin que nadie se entere. En el almacén eso es un pago que desaparece.

3 · El problema en términos técnicos

Las bases de datos tradicionales resuelven esto con candados. El primero que llega bloquea la fila o la tabla, y los demás esperan hasta que termine. Funciona, pero tiene un costo. Si un proceso se demora, todos los demás quedan parados detrás de él, y si alguien olvida soltar el candado, la tabla queda tomada.

Una tabla Iceberg no vive dentro de un motor que pueda repartir candados. Es una carpeta de archivos más un catálogo, y la escriben programas distintos que no se conocen entre ellos. Hace falta otra forma de evitar que uno pise al otro.

4 · El diagrama



Escritor A (azul) y **Escritor B** (morado). Son dos procesos, cada uno por su lado. Los dos leen la misma página vigente de la libreta y preparan su cambio sin pedirle permiso a nadie. Ninguno sabe que el otro existe.

El clavo · commit (amarillo). Al final cada uno intenta colgar su página nueva en el clavo, que es el catálogo. Colgar la página se llama hacer commit. Uno llega primero y la cuelga, y el clavo pasa a apuntar a esa página.

Llega el segundo (gris). Aquí se decide todo. Cuando llega el segundo, Iceberg no cuelga su página a ciegas. Primero revisa si la página vigente sigue siendo la que él leyó al empezar, y según la respuesta toma uno de tres caminos.

Se anota (verde). Si nadie tocó nada mientras tanto, cuelga su página y listo. Es el caso normal cuando no hay nadie más escribiendo.

Reintenta solo (verde agua). Si alguien colgó una página, pero con filas distintas a las tuyas, no hay nada que pisar. Iceberg rehace el commit solo, encima de la página nueva, y lo consigue. Nadie ve un error. Es lo que pasa en el paso 1.

Falla a propósito (rojo). Si el otro tocó la misma fila, reintentar sería escribir encima de una corrección ajena. Iceberg se niega y lanza un error, `ValidationException`. El que pierde no deja página. Es lo que pasa en el paso 2.

Ningún dato se pierde (coral). En los tres caminos nadie escribe encima de lo que no leyó. No hubo candados y nadie esperó a nadie.

5 · La solución

En el almacén, nadie le pone candado a la libreta. Cada uno lee la página de arriba, prepara su anotación aparte y, al ir a colgarla, mira si la página de arriba sigue siendo la misma que leyó. Si alguien agregó algo que no tiene que ver con lo suyo, cuelga su anotación encima y listo. Si alguien corrigió el mismo renglón que él quería corregir, no la cuelga. Vuelve, relee y decide de nuevo.

En Iceberg eso se llama **concurrency optimista**. Optimista, porque parte suponiendo que nadie va a chocar y no bloquea a nadie. La revisión se hace solo al final, en el momento del commit. Si los cambios no se pisan, Iceberg reintenta solo. Si se pisan, falla a propósito, y el proceso que perdió tiene que releer la tabla y volver a intentarlo.

6 · Los pasos

- **Paso 0.** Miras la tabla `curso.escritores_lab10`, con cuatro contribuyentes y una sola página.
- **Paso 1.** Dos hilos insertan una fila cada uno, al mismo tiempo. A ninguno le falla, y cuentas las filas y las páginas.
- **Paso 2.** Dos hilos corrigen la misma fila al mismo tiempo. Uno gana y al otro le sale `ValidationException`.
- **Paso 3.** Le pones nombre a lo que viste, la concurrency optimista, y ves qué hace un proceso de carga bien escrito cuando le rebota el commit.
- **Cierre.** Las dos situaciones lado a lado. En ninguna se perdió un dato.

Para que el laboratorio salga como en esta guía, la tabla tiene que partir con cuatro filas y una página. Si ya lo corriste antes, déjala como estaba con el comando de abajo, desde la carpeta del repositorio.

```
bash
bin/reiniciar-lab.sh 10
```

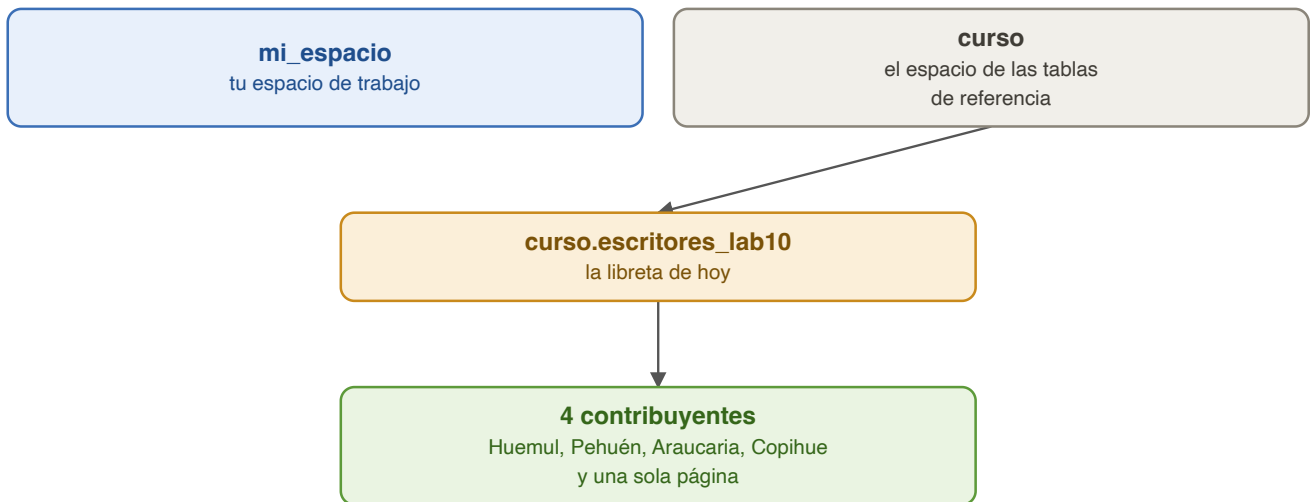
Paso 0 · La tabla compartida

Celda 0.1 · Mirar la tabla

La pregunta. ¿Qué hay en la tabla antes de que alguien escriba?

Por qué ahora. Antes de meterle mano a una tabla, se mira cómo está. Así después se puede contar qué agregó cada escritor.

En el almacén. Hoy no usas tu libreta. Usas una libreta que cuelga en otro clavo, el de `curso`, y antes de anotar la abres y lees lo que tiene.



mi_espacio (azul). Es tu espacio de trabajo, donde viven las tablas que tú creas. Hoy no se usa, porque la tabla de hoy vive en otro espacio.

curso (gris). Es el espacio de las tablas de referencia que trae el ambiente. La tabla de hoy vive ahí.

curso.escritores_lab10 (amarillo). Es la tabla de hoy. El nombre tiene dos partes, el espacio y la tabla, separadas por un punto.

4 contribuyentes (verde). La tabla parte con cuatro filas y una sola página. Así la deja `bin/reiniciar-lab.sh 10`, para que el laboratorio parta siempre igual.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor, y sirve para saber qué celda es.
- `SELECT *` pide todas las columnas. El asterisco significa todas.
- `FROM curso.escritores_lab10` dice de qué tabla leer. No hace falta `USE`, porque la tabla se nombra completa, con su espacio adelante.
- `ORDER BY rut` ordena las filas por RUT, para que salgan siempre en el mismo orden.

```
sql
%%sql
-- Celda 0.1
SELECT * FROM curso.escritores_lab10 ORDER BY rut
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:09:01 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
rut      razon_social      segmento
76171162-8 Huemul Alimentos EIRL    GRANDE
77746521-K Pehuen Logistica EIRL     MICRO
77884562-8 Araucaria Ferreteria SpA  PEQUENA
78800840-6 Copihue Maquinarias EIRL  MEDIANA
4 filas.
```

Cómo se lee. Cuatro filas, una por contribuyente, con su RUT, su razón social y su segmento. Pehuén Logística, RUT 77746521-K, está en segmento MICRO. Es la fila que se corrige en el paso 2.

Las tres primeras líneas no son errores. Salen solo en la primera celda que usa Spark, cuando arranca.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores, no todo lo que hace.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. No hace falta tocarlo. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:09:01` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

Al final, `4 filas.` es el conteo que agrega el cuaderno debajo de cada resultado.

Celda 0.2 · Contar las páginas

La pregunta. ¿Cuántas páginas tiene la libreta antes de que alguien escriba?

Por qué ahora. En el paso 1 vas a contar cuántas páginas deja cada escritor. Para eso hace falta saber desde qué número partes.

En el almacén. Es mirar la lista de páginas de la libreta antes de que alguien anote. Así, cuando cada uno cuelgue la suya, sabes cuántas se agregaron.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide tres columnas. `snapshot_id` es el número de la página, `committed_at` la hora en que se colgó, en UTC, y `operation` qué se hizo en ella.
- `FROM curso.escritores_lab10.snapshots` lee la vista de sistema `.snapshots`. Toda tabla Iceberg trae vistas de sistema que muestran su interior, y esta es la lista de páginas. El nombre tiene tres partes, el espacio, la tabla y la vista.
- `ORDER BY committed_at` ordena de la página más vieja a la más nueva.

```
sql
%%sql
-- Celda 0.2
SELECT snapshot_id, committed_at, operation
FROM curso.escritores_lab10.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
7355731461086440536  2026-09-25 18:05:23.416  append
1 fila.
```

Cómo se lee. Una sola página. Su número es 7355731461086440536 y se colgó el 25 de septiembre de 2026 a las 18:05:23 UTC, las 15:05 en Chile. `append` significa que se agregaron filas, que son los cuatro contribuyentes de la carga inicial.

La hora es la del momento en que se repuso la tabla. En tu pantalla el número y la hora van a ser otros, porque cada vez que se repone la tabla nace una página nueva.

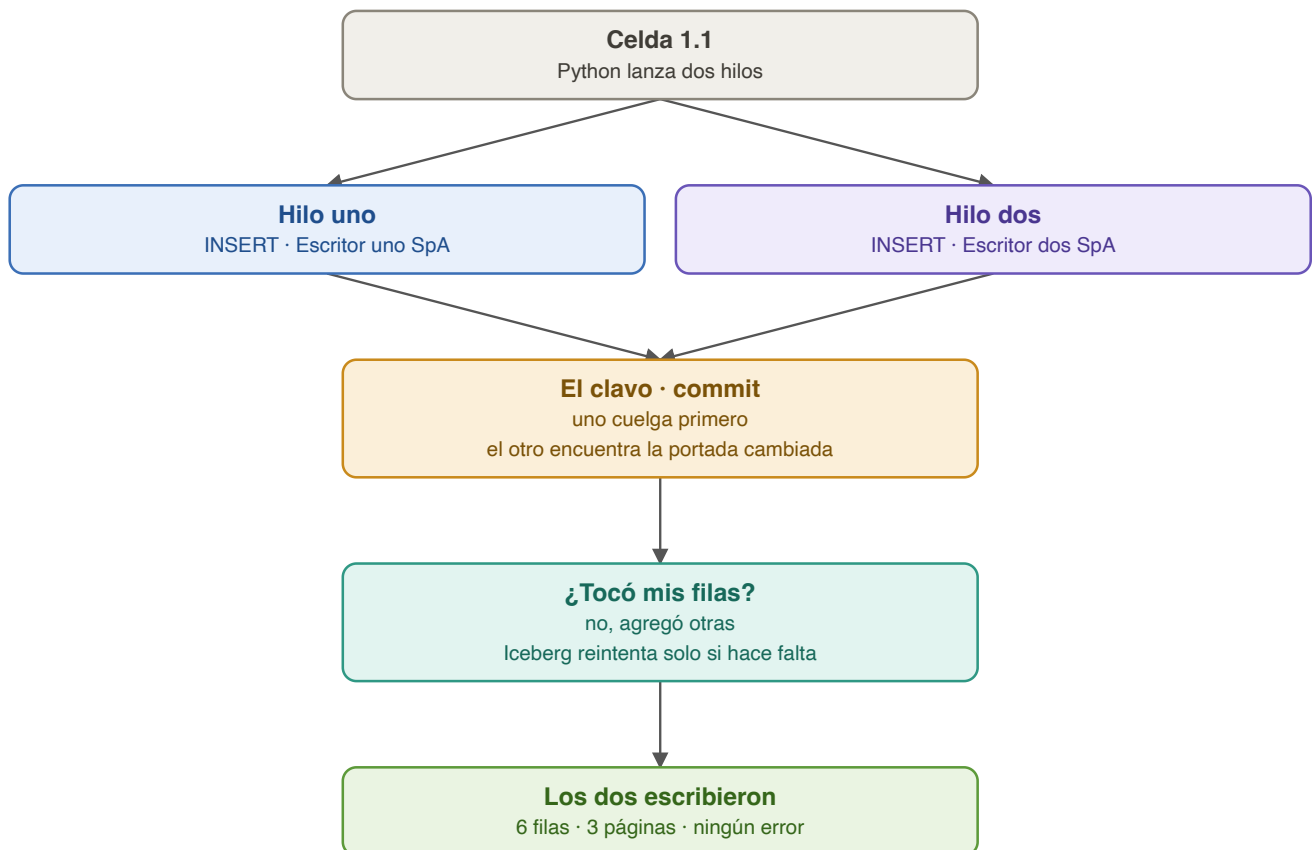
Paso 1 · Dos escritores insertan a la vez

Celda 1.1 · Los dos hilos insertan

La pregunta. ¿Qué pasa si dos escritores agregan filas a la misma tabla en el mismo instante?

Por qué ahora. Para que haya dos escritores de verdad hace falta que dos cosas escriban al mismo tiempo, y tú eres una sola persona. Esta celda resuelve eso lanzando dos hilos. Viene escrita en el cuaderno y no hay que cambiarle nada.

En el almacén. Dos personas anotan algo nuevo al final de la libreta en el mismo momento. Ninguna corrige lo que escribió la otra. Cada una agrega su propio renglón.



Celda 1.1 (gris). Es una celda de Python, por eso no lleva `%%sql`. Lanza dos tareas que corren al mismo tiempo dentro del cuaderno.

Hilo uno (azul) y **Hilo dos** (morado). Cada hilo manda su propio `INSERT` con una fila. Cada uno escribe su propio archivo Parquet, su papelito, así que hasta aquí no pueden chocar.

El clavo · commit (amarillo). Los dos intentan colgar su página casi al mismo tiempo. Uno llega primero. Si el segundo llega cuando la portada ya cambió, lo nota.

¿Tocó mis filas? (verde agua). Iceberg revisa qué cambió. El otro solo agregó una fila nueva y no tocó nada de lo que el segundo leyó. Como no hay nada que pisar, si hace falta reintenta el commit solo, encima de la página nueva.

Los dos escribieron (verde). Ninguno ve un error. Quedan seis filas y tres páginas.

La sentencia, parte por parte.

- `# Celda 1.1` es un comentario de Python. En Python los comentarios empiezan con `#`.
- `import threading` trae la biblioteca de Python que sabe correr varias tareas al mismo tiempo. Cada tarea se llama hilo.
- `def escribe(nombre)` define una función llamada `escribe`, que recibe un nombre. Es lo que va a hacer cada hilo.
- `spark.sql(...)` manda una sentencia SQL a Spark desde Python. `spark` es la conexión a Spark, y ya existe porque la celda 0.1 la creó.
- `f"INSERT INTO curso.escritores_lab10 VALUES "` y la línea que sigue arman la sentencia. La `f` antes de las comillas permite meter variables entre llaves, y `{nombre}` se reemplaza por el nombre del hilo. Las dos líneas se pegan en una sola sentencia.
- `('99000001-1', 'Escritor {nombre} SpA', 'MICRO')` es la fila. El RUT es el mismo para los dos hilos, a propósito. La tabla no tiene llave que impida repetirlo, y lo que interesa hoy es ver si entran las dos filas, no el RUT.
- `print(nombre, "escribio")` avisa en pantalla que ese hilo terminó su `INSERT`.
- `hilos = [threading.Thread(target=escribe, args=(n,)) for n in ("uno", "dos")]` crea dos hilos, uno llamado `uno` y otro `dos`. `target` dice qué función corre cada hilo y `args` con qué nombre. La coma en `(n,)` hace una tupla de un solo valor, que es lo que pide `args`. Una tupla es una secuencia fija de valores entre paréntesis, y sin la coma `(n)` sería solo `n` entre paréntesis, no una tupla.
- `for h in hilos: h.start()` echa a andar los dos hilos, uno detrás del otro, con microsegundos de diferencia.
- `for h in hilos: h.join()` espera a que los dos terminen antes de seguir.
- `print("los dos terminaron")` avisa que la celda terminó.

```
python
# Celda 1.1
import threading

def escribe(nombre):
    spark.sql(
        f"INSERT INTO curso.escritores_lab10 VALUES "
        f"('99000001-1', 'Escritor {nombre} SpA', 'MICRO')"
    )
    print(nombre, "escribio")

hilos = [threading.Thread(target=escribe, args=(n,)) for n in ("uno", "dos")]
for h in hilos:
    h.start()
for h in hilos:
    h.join()

print("los dos terminaron")
```

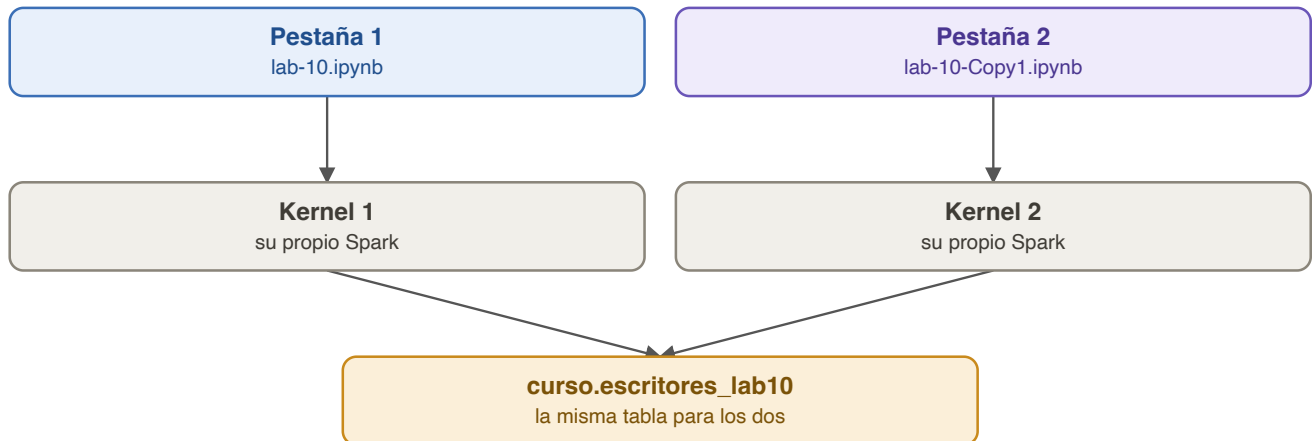
Lo que sale en pantalla.

```
text
dos escribio
uno escribio
los dos terminaron
```

Cómo se lee. Los dos hilos escribieron y ninguno falló. El hilo `dos` terminó antes que el `uno`, aunque se lanzó después. El orden de llegada no se puede prever, y en tu pantalla puede salir al revés. `escribio` va sin tilde porque es texto dentro del código.

A mano, con dos pestañas

También se puede hacer a mano. Para que sean dos escritores de verdad, cada pestaña necesita su propio kernel, que es el proceso de Python que ejecuta las celdas y que tiene su propio Spark. El mismo cuaderno abierto dos veces comparte un solo kernel, y las dos celdas se ejecutarían una detrás de la otra en el mismo Spark, como un solo escritor.



Pestaña 1 (azul). Es el cuaderno del laboratorio, `lab-10.ipynb`, abierto como siempre.

Pestaña 2 (morado). Es una copia del cuaderno. Se hace en el explorador de archivos de JupyterLab, con clic derecho sobre `lab-10.ipynb` y **Duplicate**, y la copia se llama `lab-10-Copy1.ipynb`. Se abre en otra pestaña del navegador.

Kernel 1 y Kernel 2 (gris). Cada cuaderno tiene su propio kernel y su propio Spark. Por eso son dos escritores distintos.

curso.escritores_lab10 (amarillo). Los dos escriben en la misma tabla.

En una celda nueva de cada pestaña escribe el mismo `INSERT`, cada uno con un nombre distinto, y ejecútalas una detrás de la otra, lo más rápido que puedas. Por ejemplo, en la pestaña 1 va la sentencia de abajo con `Escritor pestaña uno SpA`, y en la pestaña 2 con `Escritor pestaña dos SpA`.

```
sql
%%sql
INSERT INTO curso.escritores_lab10 VALUES
('99000001-1', 'Escritor pestaña uno SpA', 'MICRO')
```

Las dos van a decir `Listo`. La sentencia se ejecutó. Si lo haces, la tabla queda con dos filas más de las que muestra esta guía, y las celdas que siguen van a dar otros números. Para volver a partir, corre `bin/reiniciar-lab.sh 10`.

Celda 1.2 · Contar las filas

La pregunta. ¿Entraron las dos filas, o alguna se perdió?

Por qué ahora. Que no haya fallado no quiere decir que haya quedado bien. Hay que contar.

En el almacén. Es contar los renglones de la libreta después de que los dos anotaron. Tienen que estar los de antes y uno más por cada persona que anotó.

La sentencia, parte por parte.

- `count(*)` cuenta las filas. El asterisco significa todas las filas, sin fijarse en ninguna columna.
- `AS filas` le pone nombre a la columna del resultado.
- `FROM curso.escritores_lab10` es la tabla de hoy.

```
sql
%%sql
-- Celda 1.2
SELECT count(*) AS filas FROM curso.escritores_lab10
```

Lo que sale en pantalla.

```
text
filas
6
1 fila.
```

Cómo se lee. Seis filas. Son las cuatro de la carga inicial más una por cada hilo. Si saliera un número menor, alguna escritura se habría perdido. Salen seis, así que entraron las dos aunque llegaron al mismo tiempo.

Celda 1.3 · Mirar las páginas

La pregunta. ¿Cómo quedó la historia de la libreta después de los dos `INSERT` ?

Por qué ahora. Ya sabes que las filas entraron. Ahora ves cómo quedaron anotadas. Si cada escritor colgó su página, tiene que haber una más por cada uno.

En el almacén. Es mirar de nuevo la lista de páginas. Antes había una sola, y ahora tiene que haber una por cada persona que anotó.

La sentencia, parte por parte. Es la misma de la celda 0.2, y se lee igual.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué se hizo.
- `FROM curso.escritores_lab10.snapshots` es la vista de sistema con la lista de páginas.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 1.3
SELECT snapshot_id, committed_at, operation
FROM curso.escritores_lab10.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
7355731461086440536 2026-09-25 18:05:23.416 append
6046838079518803208 2026-09-25 18:09:09.544 append
5186261370086278124 2026-09-25 18:09:09.858 append
3 filas.
```

Cómo se lee. Tres páginas, las tres `append`.

- La primera, `7355731461086440536`, es la carga inicial, la misma de la celda 0.2.
- La segunda, `6046838079518803208`, se colgó a las 18:09:09.544 UTC, las 15:09 en Chile. Es la del hilo que llegó primero al clavo.
- La tercera, `5186261370086278124`, se colgó a las 18:09:09.858 UTC, 314 milésimas de segundo después. Es la del segundo hilo.

No hay ninguna página que diga reintento ni error. Si el segundo hilo tuvo que reintentar, no queda rastro, porque el reintento pasa dentro del commit y solo se anota la página que al final se colgó. Lo que sí se ve es lo importante. Entraron los dos y ninguno se perdió.

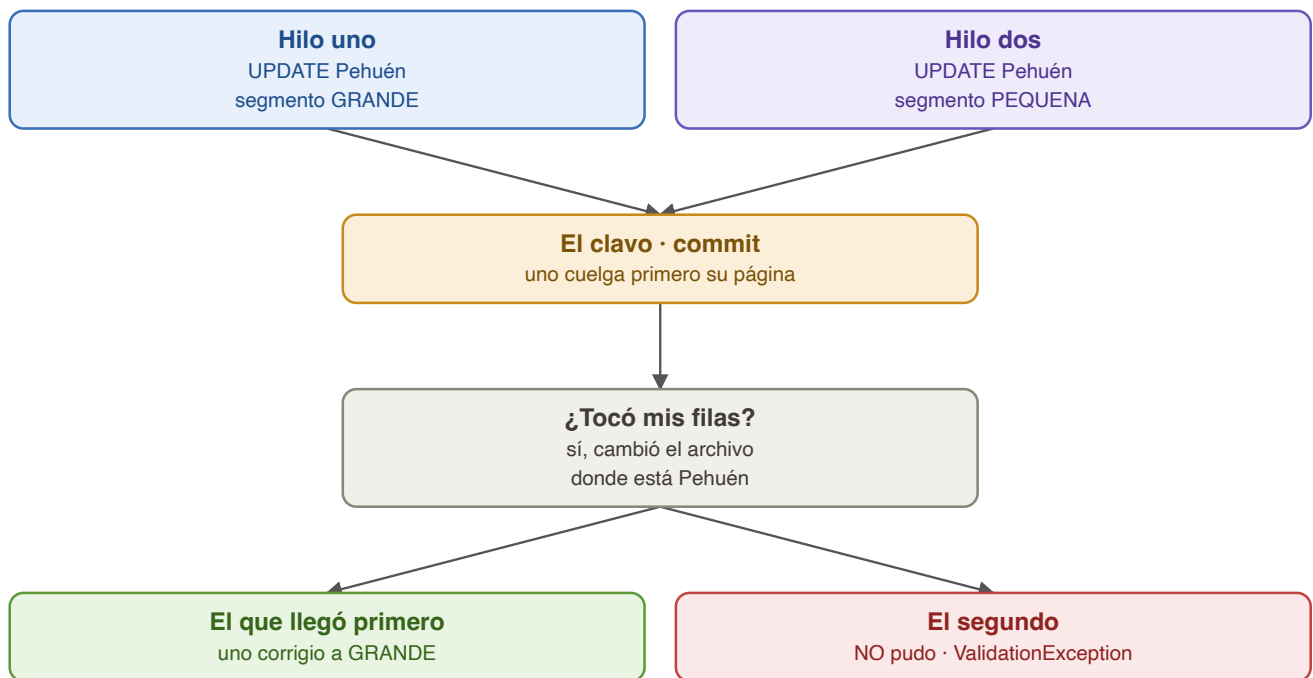
Paso 2 · Los dos corrigen la misma fila

Celda 2.1 · Los dos hilos corrigen a Pehuén

La pregunta. ¿Qué pasa si dos procesos corrigen la misma fila en el mismo instante?

Por qué ahora. En el paso 1 los dos agregaron cosas distintas. Ahora los dos quieren cambiar lo mismo, la fila de Pehuén, y cada uno le pone un segmento distinto. Es el caso que importa en producción. Esta celda también viene escrita en el cuaderno.

En el almacén. Dos empleados quieren corregir el mismo renglón de la libreta, el de Pehuén, al mismo tiempo. Los dos leen la misma página, los dos preparan su corrección y los dos corren a colgarla.



Hilo uno (azul) y **Hilo dos** (morado). Los dos leen la misma página y los dos preparan un `UPDATE` sobre la fila de Pehuén. El uno le pone `GRANDE` y el dos `PEQUENA`.

El clavo · commit (amarillo). Para cambiar una fila, Iceberg no tacha. Escribe un papelito nuevo con la fila ya corregida y deja de usar el viejo. Los dos hilos reescriben el mismo papelito, el de Pehuén, y uno llega primero al clavo.

¿Tocó mis filas? (gris). Esta vez la respuesta es sí. El primero cambió el mismo papelito que el segundo leyó. Reintentar sería escribir encima de una corrección ajena, así que Iceberg no reintentará.

El que llegó primero (verde). Su corrección queda en la tabla.

El segundo (rojo). Recibe `ValidationException` y no deja página. No se sabe de antemano cuál de los dos va a ganar.

La sentencia, parte por parte.

- `import threading` trae otra vez la biblioteca de los hilos.
- `def corrige(nombre, segmento)` define la función que corre cada hilo. Recibe el nombre del hilo y el segmento que le va a poner a Pehuén.
- `try:` abre un bloque que puede fallar. Si falla, Python salta al `except` en vez de detener la celda.
- `spark.sql(f"UPDATE curso.escritores_lab10 SET segmento = '{segmento}' " f"WHERE rut = '77746521-K'")` es la corrección. `UPDATE` cambia filas que ya existen, `SET segmento = '{segmento}'` pone el segmento que recibió el hilo y `WHERE rut = '77746521-K'` apunta solo a Pehuén, la misma fila para los dos.
- `print(nombre, "corrigo a", segmento)` avisa que ese hilo logró su corrección.
- `except Exception as error:` atrapa cualquier error y lo guarda en la variable `error`.
- `lineas = str(error).splitlines()` convierte el error en texto y lo parte en líneas. El error de Spark es largo, con decenas de líneas.
- `print(nombre, "NO pudo:", lineas[0][:90])` muestra la primera línea, recortada a 90 caracteres.
- `causa = [l.strip() for l in lineas if "ValidationException" in l]` busca entre todas las líneas las que dicen `ValidationException`, que es donde Iceberg explica la causa. `strip()` les saca los espacios de los bordes.
- `if causa: print(causa[0])` muestra la primera de esas líneas, si hay alguna.
- `hilos = [threading.Thread(target=corrige, args=("uno", "GRANDE")), threading.Thread(target=corrige, args=("dos", "PEQUENA"))]` crea los dos hilos, cada uno con su nombre y su segmento.
- `start()` los echa a andar y `join()` espera a que terminen, igual que en la celda 1.1.

```
python
# Celda 2.1
import threading

def corrige(nombre, segmento):
    try:
        spark.sql(
            f"UPDATE curso.escritores_lab10 SET segmento = '{segmento}' "
            f"WHERE rut = '77746521-K'"
        )
        print(nombre, "corrigio a", segmento)
    except Exception as error:
        lineas = str(error).splitlines()
        print(nombre, "NO pudo:", lineas[0][:90])
        causa = [l.strip() for l in lineas if "ValidationException" in l]
        if causa:
            print(causa[0])

hilos = [
    threading.Thread(target=corrige, args=("uno", "GRANDE")),
    threading.Thread(target=corrige, args=("dos", "PEQUENA")),
]
for h in hilos:
    h.start()
for h in hilos:
    h.join()

print("los dos terminaron")
```

Lo que sale en pantalla.

```
text
uno corrigio a GRANDE
26/09/25 18:09:12 ERROR ReplaceDataExec: Data source write support
IcebergBatchWrite(table=spark_catalog.curso.escritores_lab10, format=PARQUET) is aborting.
26/09/25 18:09:12 ERROR ReplaceDataExec: Data source write support
IcebergBatchWrite(table=spark_catalog.curso.escritores_lab10, format=PARQUET) aborted.
dos NO pudo: An error occurred while calling o39.sql.
Caused by: org.apache.iceberg.exceptions.ValidationException: Found conflicting files that can
contain records matching ref(name="rut") == "77746521-K":
[hdfs://namenode:8020/warehouse/iceberg/curso.db/escritores_lab10/data/00003-11-b5262010-d20e-
4613-bd26-2794f5407040-00001.parquet]
los dos terminaron
```

Cómo se lee. Primero lo simple. El hilo `uno` ganó y dejó a Pehuén en `GRANDE`. El hilo `dos` no pudo, e Iceberg dice por qué.

Ahora cada línea.

- `uno` corrigio a `GRANDE` es el hilo que llegó primero. Su corrección entró.
- Las dos líneas `ERROR ReplaceDataExec` son Spark avisando que descarta la escritura del hilo que perdió. `ReplaceDataExec` es la parte de Spark que reemplaza el papelito donde vive la fila. `IcebergBatchWrite` es la escritura hacia Iceberg, `table=spark_catalog.curso.escritores_lab10` es la tabla, con `spark_catalog` como nombre del catálogo, y `format=PARQUET` el formato de los papelitos. `is aborting` y después `aborted` significan que la está abortando y que ya la abortó. El papelito que alcanzó a escribir el perdedor se borra.
- `dos` NO pudo: `An error occurred while calling o39.sql.` es la primera línea del error. `o39.sql` es el nombre interno de la llamada que Python le hizo a Spark, y no importa.
- `Caused by: org.apache.iceberg.exceptions.ValidationException` es la causa. `Caused by` quiere decir causado por, y `ValidationException` es el error que lanza Iceberg cuando la revisión del commit no pasa.
- `Found conflicting files that can contain records matching ref(name="rut") == "77746521-K"` quiere decir que encontró archivos en conflicto que pueden contener filas con RUT `77746521-K`. `ref(name="rut")` es la columna `rut` del filtro del `UPDATE`.
- Entre corchetes va el archivo en conflicto, el papelito nuevo que dejó el ganador. `hdfs://namenode:8020` es el almacenamiento, `/warehouse/iceberg/curso.db/escritores_lab10/data/` la carpeta de datos de la tabla y `00003-11-b5262010-d20e-4613-bd26-2794f5407040-00001.parquet` el nombre del archivo, que Spark arma con el número de la tarea y un identificador al azar.
- `los dos terminaron` sale cuando terminan los dos hilos, ganara quien ganara.

Las dos líneas `ERROR` salen en rojo en el cuaderno. No son un problema de la celda. Son el aviso de que Iceberg hizo su trabajo.

Celda 2.2 · Ver quién quedó

La pregunta. Después de dos correcciones sobre la misma fila, ¿cuál quedó?

Por qué ahora. La celda anterior dijo quién ganó. Esta lo comprueba en la tabla. Tiene que haber un solo segmento, el del ganador, y nunca una mezcla.

En el almacén. Es ir a leer el renglón de Pehuén en la libreta. Tiene que decir una sola cosa, la de quien colgó su corrección.

La sentencia, parte por parte.

- `SELECT *` trae todas las columnas.
- `FROM curso.escritores_lab10` es la tabla de hoy.
- `WHERE rut = '77746521-K'` se queda solo con la fila de Pehuén.

```
sql
%%sql
-- Celda 2.2
SELECT * FROM curso.escritores_lab10 WHERE rut = '77746521-K'
```

Lo que sale en pantalla.

```
text
rut      razon_social      segmento
77746521-K Pehuen Logistica EIRL  GRANDE
1 fila.
```

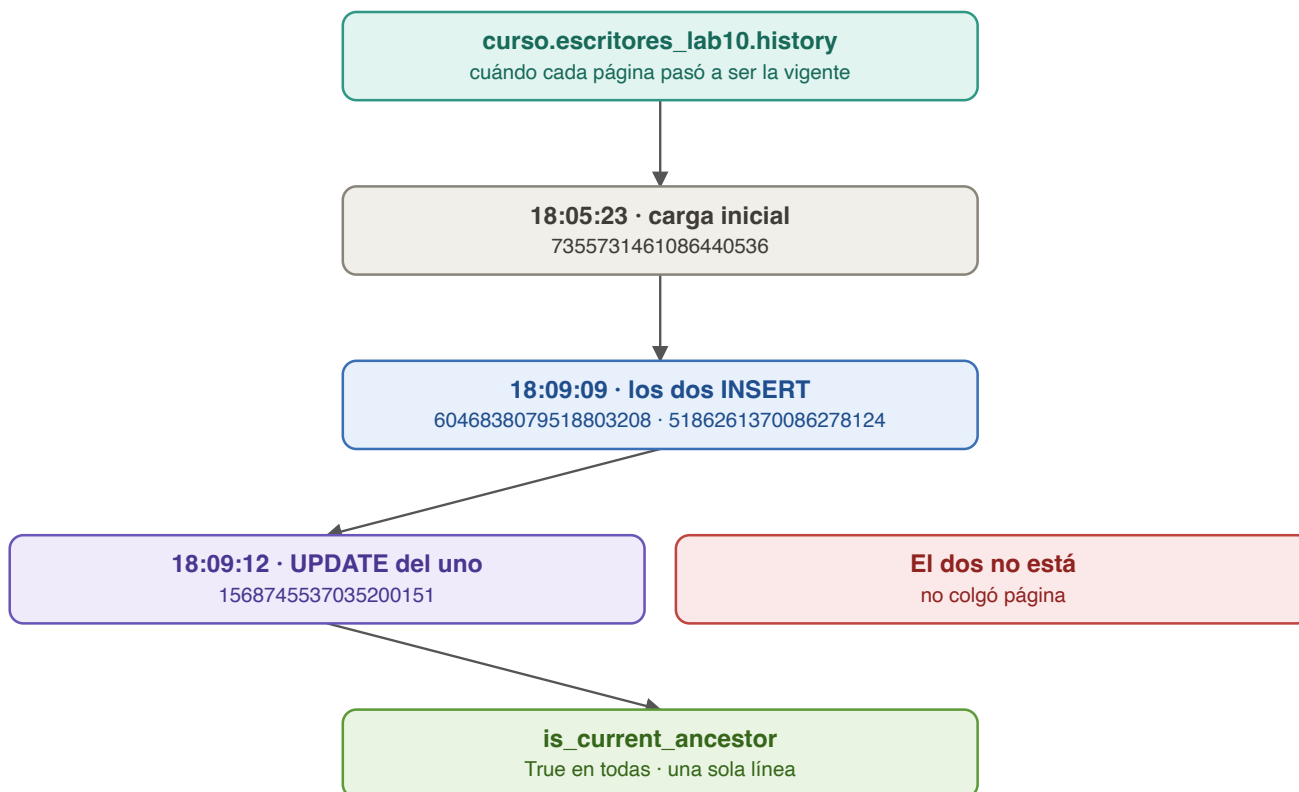
Cómo se lee. Una fila. Pehuén quedó en **GRANDE**, que es lo que puso el hilo **uno**, el ganador. El **PEQUENA** del hilo **dos** no aparece por ninguna parte. Al empezar el laboratorio Pehuén estaba en **MICRO**.

Celda 2.3 · La historia completa

La pregunta. ¿Qué escrituras lograron entrar a la libreta, y en qué orden?

Por qué ahora. La fila dice quién quedó al final. La historia dice quién logró escribir y cuándo. El que perdió no aparece, porque nunca colgó su página.

En el almacén. Es mirar el registro de cuándo cada página pasó a ser la vigente. Solo aparece quien alcanzó a colgar la suya.



curso.escritores_lab10.history (verde agua). Es otra vista de sistema. No lista las páginas, sino los momentos en que cada una pasó a ser la vigente.

18:05:23 · carga inicial (gris). Es la página de la reposición, con los cuatro contribuyentes. Son las 15:05 en Chile.

18:09:09 · los dos INSERT (azul). Las dos filas del paso 1, cada una con su página.

18:09:12 · UPDATE del uno (morado). Es la corrección del ganador.

El dos no está (rojo). No aparece en ninguna parte. Perdió, Iceberg se negó a colgar su página y por eso no hay nada que mostrar.

is_current_ancestor (verde). Esa columna dice si la página forma parte del camino que lleva a la vigente de hoy. Sale **True** en todas, así que la historia es una sola línea, sin páginas abandonadas.

La sentencia, parte por parte.

- **made_current_at** es cuándo esa página pasó a ser la vigente, en UTC.
- **snapshot_id** es el número de la página.
- **is_current_ancestor** dice si la página está en el camino que llega a la vigente.
- **FROM curso.escritores_lab10.history** es la vista de sistema, con sus tres partes.
- **ORDER BY made_current_at** ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 2.3
SELECT made_current_at, snapshot_id, is_current_ancestor
FROM curso.escritores_lab10.history
ORDER BY made_current_at
```

Lo que sale en pantalla.

```
text
made_current_at      snapshot_id          is_current_ancestor
2026-09-25 18:05:23.416 7355731461086440536 True
2026-09-25 18:09:09.544 6046838079518803208 True
2026-09-25 18:09:09.858 5186261370086278124 True
2026-09-25 18:09:12.000 1568745537035200151 True
4 filas.
```

Cómo se lee. Cuatro momentos, y los cuatro con **True**.

- 18:05:23 UTC, la carga inicial.
- 18:09:09.544 y 18:09:09.858 UTC, los dos **INSERT** del paso 1.
- 18:09:12 UTC, las 15:09 en Chile, el **UPDATE** del hilo **uno**, la página **1568745537035200151**.

No hay una quinta fila para el hilo **dos**. Su corrección nunca llegó a la libreta.

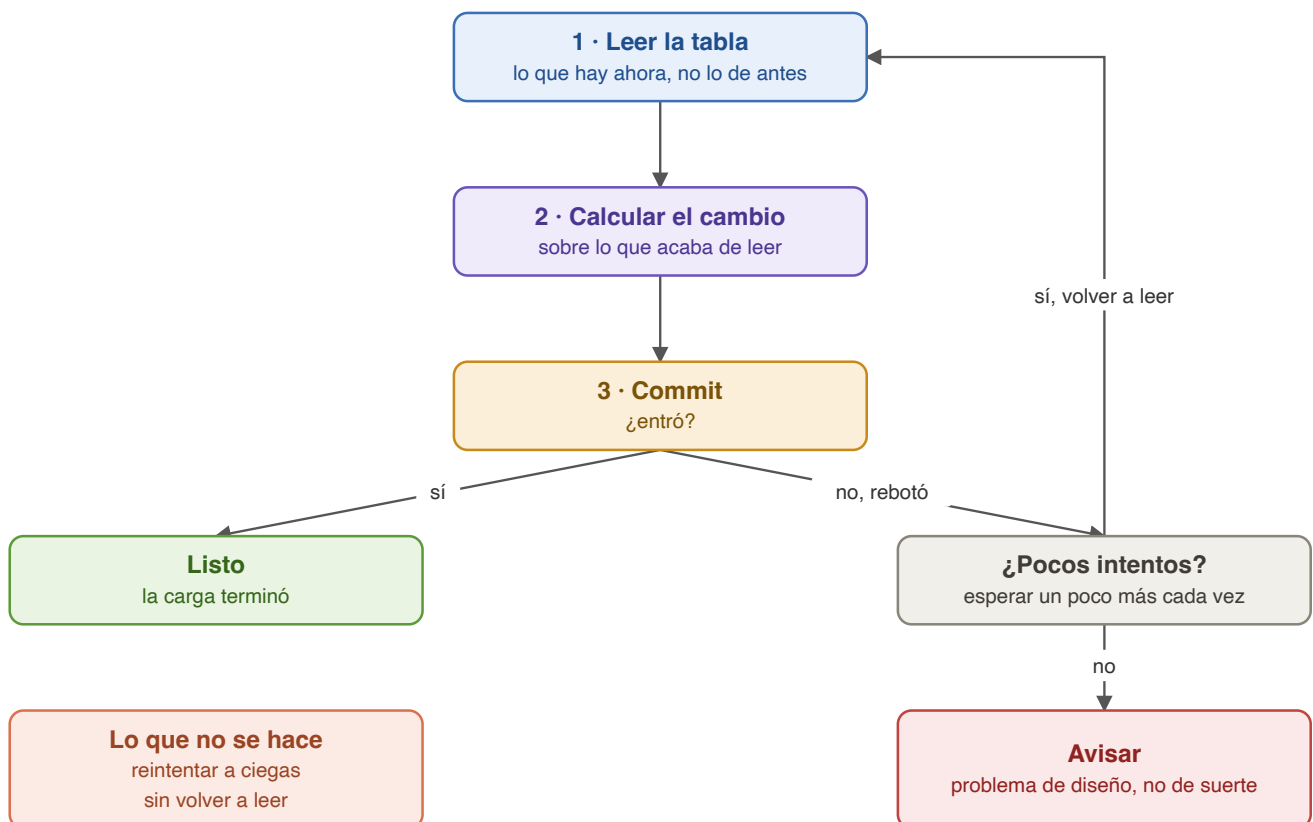
Paso 3 · Concurrencia optimista

Este paso no tiene celdas que ejecutar. Es texto en el cuaderno, y aquí va explicado.

La pregunta. ¿Cómo se llama lo que acabas de ver, y qué tiene que hacer un proceso de carga cuando le rebota el commit?

Por qué ahora. Viste dos experimentos con resultados distintos. Ahora les pones nombre y sacas la lección para producción, porque el error del paso 2 es el que le va a aparecer a un proceso nocturno el día que dos cargas choquen.

En el almacén. Nadie le pone candado a la libreta. Cada uno prepara su anotación aparte y solo al colgarla revisa si alguien cambió lo mismo. Si le rebotan la hoja, no la cuelga a la fuerza. Vuelve, relee la libreta y rehace su anotación sobre lo que hay ahora.



1 · Leer la tabla (azul). Todo proceso de carga parte leyendo la tabla como está en ese momento. Si reintenta, vuelve a leerla, porque lo que había antes ya cambió.

2 · Calcular el cambio (morado). Prepara su cambio sobre lo que acaba de leer, sin pedirle permiso a nadie y sin poner candados.

3 · Commit (amarillo). Intenta colgar su página. Iceberg ya resolvió solo el caso de las filas nuevas que no se pisan. Si igual rebota, es porque alguien cambió lo mismo, como en el paso 2.

Listo (verde). Si entró, la carga terminó.

¿Pocos intentos? (gris). Si rebotó, un error de commit no es un aviso que se pueda ignorar. El proceso vuelve al paso 1, relee y rehace su cálculo. Reintenta unas pocas veces, esperando un poco más entre un intento y el siguiente, para no volver a chocar con el mismo proceso.

Avisar (rojo). Si sigue rebotando después de varios intentos, avisa. Que dos procesos peleen siempre por las mismas filas es un problema de diseño, no de mala suerte. Se arregla más arriba, por ejemplo separando las cargas por partición para que no toquen los mismos archivos.

Lo que no se hace (coral). Reintentar a ciegas, mandando de nuevo el mismo cambio sin volver a leer. Eso sería escribir encima de lo que no se leyó, que es justo lo que Iceberg acaba de evitar. Tampoco se sigue como si nada hubiera pasado.

Esto se llama concurrencia optimista. Iceberg parte suponiendo que nadie va a chocar, no bloquea a nadie y revisa solo al final. Si nadie tocó nada, se anota. Si alguien anotó algo que no se pisa con lo tuyo, Iceberg reintentará solo y nadie se entera. Si alguien tocó los mismos archivos, falla a propósito. El tercer caso es el que importa en producción, y es el que casi nadie espera la primera vez.

Preguntas frecuentes

¿Quién hace los reintentos?

Son dos reintentos distintos, y cada uno lo hace alguien distinto.

El reintento de Iceberg. Lo hace Iceberg solo, por dentro, en la biblioteca de Iceberg que Spark lleva adentro. Es para las filas nuevas que no se pisan. Cuando un escritor va a colgar su página y encuentra que otro colgó una antes, Iceberg revisa qué cambió. Si el otro solo agregó filas y no tocó nada de lo que él leyó, vuelve a intentar colgar su página encima de la nueva. No aparece en pantalla ni deja rastro en la historia. Se configura por tabla, con la propiedad `commit.retry.num-retries`, que por defecto son 4 reintentos, con una espera que parte en 100 milisegundos y va creciendo.

El reintento del proceso de carga. Lo tiene que programar quien escribe el proceso, y nadie lo hace por él. Cuando el choque es por la misma fila, como en el paso 2, Iceberg no reintenta, porque reintentar sería escribir encima de otro. Lanza el error y se detiene. Desde ahí, reintentar es trabajo del programa que lanzó la escritura o del orquestador que lo ejecuta. Ese programa tiene que atrapar el error, volver a leer la tabla, rehacer su cambio y probar de nuevo. Es lo que muestra el diagrama del paso 3.

Qué se vio y qué no. El reintento de Iceberg no se puede ver. En el paso 1 las dos páginas quedaron a 314 milésimas de segundo una de otra. Los dos hilos partieron juntos, así que es probable que el segundo haya encontrado la portada cambiada y haya reintentado, pero en pantalla no queda rastro. Lo único visible es que entraron las dos filas. El reintento del proceso de carga se vio al revés, por su ausencia. La celda 2.1 no tiene reintento. Cuando el hilo `dos` falló, imprimió `NO pudo` y se quedó ahí. Si fuera un proceso nocturno escrito así, su corrección se perdería y nadie se enteraría hasta el otro día. Iceberg hizo su parte, que era no dejarlo escribir encima. Lo que falta es el programa que atrapa el error y vuelve a leer.

	Quién reintenta	Cuándo	Se ve
Filas nuevas que no se pisan	Iceberg, solo	dentro del commit	no
La misma fila	nadie, si no lo programas	después del error	sí, el error

¿Por qué ganó el hilo uno?

Porque llegó primero al clavo. No hay ninguna regla que favorezca a uno u otro. Depende de cuál termina antes de preparar su cambio, y eso varía de una vez a otra. Si repites el laboratorio puede ganar el `dos`, y entonces Pehuén queda en `PEQUENA`.

¿Por qué hace falta una copia del cuaderno para las dos pestañas?

Porque JupyterLab asocia cada cuaderno a un solo kernel. Si abres el mismo `lab-10.ipynb` en dos pestañas, las dos usan el mismo kernel y el mismo Spark, y las celdas se ejecutan una detrás de la otra. Sería un solo escritor. La copia `lab-10-Copy1.ipynb` es otro cuaderno, con su propio kernel y su propio Spark, y por eso sí son dos escritores.

Si lo hago a mano con dos pestañas, ¿me va a salir el error?

Con los `INSERT` no, nunca. Con dos `UPDATE` sobre Pehuén, solo si se cruzan en el tiempo. Un `UPDATE` demora un par de segundos. Si el segundo parte antes de que el primero cuelgue su página, chocan, y al segundo le sale el mismo `ValidationException`. Si parte después, lee la página ya corregida y la corrige de nuevo, los dos terminan bien y queda el segmento del último. En los dos casos nadie escribe encima de lo que no leyó. Por eso el cuaderno usa hilos, porque con ellos el choque se ve seguro.

¿Por qué la tabla acepta dos filas con el mismo RUT?

Porque una tabla Iceberg no tiene llave primaria que lo impida. Si hace falta que el RUT no se repita, lo tiene que cuidar el proceso que carga, por ejemplo con `MERGE INTO`. En este laboratorio el RUT repetido es a propósito, porque lo que interesa es ver si entran las dos escrituras.

¿Iceberg bloquea la tabla mientras alguien escribe?

No. Ningún escritor espera a otro. Cada uno escribe sus papelitos por su cuenta y la revisión se hace solo en el commit, que es un instante. Por eso se llama optimista.

¿Qué pasa con el papelito que alcanzó a escribir el que perdió?

Spark lo borra al abortar la escritura. Es lo que dicen las líneas `is aborting` y `aborted`. Si un proceso se cae de golpe, sin alcanzar a abortar, el papelito puede quedar en la carpeta sin que ninguna página lo nombre. Ese es un archivo huérfano, y se limpia con las rutinas de mantención.

¿Qué pasa si corro el cuaderno dos veces sin reiniciar?

La tabla sigue acumulando. La segunda vez vas a ver ocho filas en vez de seis, más páginas en la historia, y Pehuén ya no parte en `MICRO`. Para que salga como en esta guía, antes corre `bin/reiniciar-lab.sh 10`.

¿Por qué las horas no calzan con mi reloj?

Porque Spark muestra las horas en UTC, la hora universal. Chile en septiembre está tres horas atrás, así que las 18:09 UTC son las 15:09 en Chile.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Viste dos situaciones que se parecen pero no son iguales. Agregar cosas nuevas al mismo tiempo, y corregir lo mismo al mismo tiempo.

En el almacén. Dos personas anotaron en la misma libreta sin candados y sin esperar turno. Las dos que agregaron renglones nuevos quedaron. De las dos que corrigieron el mismo renglón quedó una sola, y la otra se enteró de que no pudo. Nadie escribió encima de lo que no había leído.



Paso 1 · cada hilo agrega una fila (azul). Cada hilo agregó una fila distinta con `INSERT`. Ninguno tocó lo del otro.

Paso 2 · los dos corrigen la misma (morado). Los dos hilos corrigieron la fila de Pehuén con `UPDATE`.

Iceberg reintentando solo (verde agua). En el paso 1 no hay nada que pisar. Si hace falta, Iceberg reintentando el commit solo, por dentro y sin avisar.

Iceberg falla a propósito (rojo). En el paso 2, reintentar sería escribir encima de una corrección ajena. Iceberg se niega y lanza `ValidationException`. Desde ahí, reintentar le toca al proceso que escribió, releyendo la tabla.

Qué se vio en el paso 1 y Qué se vio en el paso 2 (gris). En el paso 1 no se vio nada, seis filas y una página por escritor. En el paso 2 el hilo `uno` corrigió y el `dos` no pudo, y el que perdió no dejó página.

Ningún dato perdido (verde). En el paso 1 están todas las filas. En el paso 2 quedó la corrección del que ganó, y el que perdió se enteró de que no pudo.

Los números de la solución ejecutada.

Momento	Filas	Páginas	Qué pasó
Al empezar	4	1	la carga inicial
Después de los dos INSERT	6	3	entraron los dos, nadie falló
Después de los dos UPDATE	6	4	el uno dejó a Pehuén en GRANDE , el dos recibió ValidationException

Eso permite que varios procesos escriban en la misma tabla sin coordinarse. No hay un coordinador, hay una regla, y la regla es que nadie escribe encima de lo que no leyó.